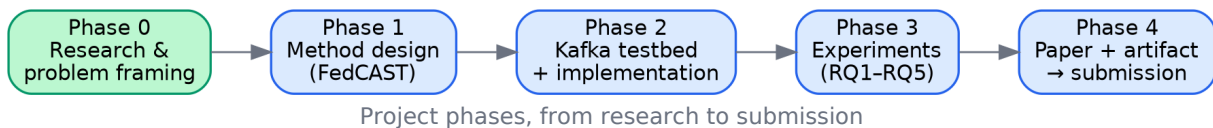


Budget-Aware Federated Stream Clustering over Apache Kafka

under Non-IID Data Distribution and Network Constraints

Project summary: requirements, research plan, architecture and progress

Course: Stream Processing Analytics · Repository: github.com/kushsriv/spa · 27 September 2026



Status: Phase 0 (research and problem framing) is complete and the Kafka testbed is running. The method, proofs and results described here are a **plan**. Nothing has been implemented or measured beyond the Kafka setup yet.

Contents

- 1. Your requirements (what you asked for)
- 2. Work done so far
- 3. Research summary and gap
- 4. Problem statement and research questions
- 5. Objectives
- 6. Methodology, with architecture and flowcharts
- 7. Experimental plan
- 8. Expected outcomes
- 9. Target venues
- 10. Roadmap and timeline
- 11. What your team needs to do next
- 12. Key references

1. Your requirements

These are the requirements you gave me across our conversation, collected in one place.

Requirement	What you said	How the plan meets it
Course	Stream Processing Analytics	Stream-native design: online micro-clusters, Kafka topics, latency and consumer-lag metrics
Topic	Cost-Aware Federated Stream Clustering under Non-IID Data Distribution and Network Constraints	Kept; sharpened into a budget-constrained problem with a new method (working name FedCAST)
Kafka	Using Kafka is essential; the most important part	Kafka is part of the algorithm, not just a pipe: keyed ordering, log compaction, offset replay for recovery, global-model broadcast, idempotent producers
Paper quality	Q1-level or above research paper	Theory (two propositions), a real system, 5 research questions, 6+ baselines, 5 datasets, statistical tests
Research first	Extensive research: problem statement, objectives, methodology, expected outcomes	Done: docs/00_research_proposal.md and this PDF
Coding	Claude writes all the code	Claude implements everything; the team reviews, runs experiments and checks references
Timeline	About one month	Four-week plan with decision gates (Section 10)
Hardware	Laptops only, less than 4 GB of free RAM	Whole testbed under 1 GB (measured): Kafka with a 256 MB heap, with or without Docker; all edge nodes in one Python process; in-app network emulation; no Flink or Spark
Venues	IEEE, ACM, Elsevier and Springer are all targets	Shortlist across all four publishers (Section 9)
Repository	New repo called spa	github.com/kushsriv/spa; Phase 0 pushed to main

2. Work done so far

#	Item	Status	Where
1	Literature search: distributed stream clustering, federated clustering, federated learning on streams, event-triggered communication, Kafka for ML, private stream clustering	Done	Proposal §2
2	Gap analysis against the closest prior work (Cormode 2007, DGClust 2011, Tran 2013, k-FED 2021, AFCL 2025, FedCluLearn 2025, DFAS 2025, Kafka-ML)	Done	Proposal §2.8
3	Problem statement, 5 research questions, 6 objectives	Done	Proposal §3–4
4	Method design: staleness score, budget-aware send rule, aggregation, Kafka protocol	Designed, not coded	Proposal §5
5	Experimental design: datasets, non-IID schemes, baselines, metrics, statistics	Done	Proposal §6
6	Target venue shortlist and a four-week roadmap	Done	Proposal §9, 01_roadmap.md
7	Kafka testbed: KRaft single broker, 5 topics (2 log-compacted); Docker Compose version and a no-Docker script	Done and tested	docker/, scripts/kafka_native.sh
7b	Low-memory redesign: Kafka measured at 317–423 MB (no Docker) and at most 512 MB (Docker, capped) under a 300k-message load test	Measured	docs/02_low_memory_setup.md
8	Tested: Kafka started, topics created, idempotent produce and consume round trip succeeded from Python	Verified	In this session
9	Architecture diagram, edge and coordinator flowcharts, Gantt chart	Done	docs/figures/
10	Pushed to github.com/kushsriv/spa (branch main)	Done	Git

Not done yet: any clustering code, the baselines, the experiments, the proofs and the paper itself. Some references were taken from search results because Springer and arXiv pages were blocked in my environment; they are flagged for your team to check.

3. Research summary and gap

Edge devices produce fast, unlabeled streams. Each device sees a different slice of the data (non-IID), over slow or unreliable links. We want one continuously updated global clustering, without raw data leaving any device, and within a byte budget per device.

Line of work	Examples	What is missing for our setting
Stream clustering (local building block)	CluStream (2003), DenStream (2006), DBSTREAM (2016)	Single site only; but their micro-cluster summaries (CF vectors) are additive and cheap to send, so we reuse them
Distributed stream clustering	Cormode et al., ICDE 2007; DGClust, 2011; Tran, 2013	Fixed thresholds or periods; no byte budget; no link awareness; ignore non-IID data; no real messaging system
Federated clustering	k-FED (ICML 2021), FFCM (2022), AFCL (AAAI 2025)	Static datasets only: no streams, no drift
Federated learning on streams	FedCluLearn (ECML-PKDD 2025), DFAS (WSDM 2025), Silva et al. (2022)	Supervised learning or anomaly detection, not clustering the data under a budget
Event-triggered communication	EventGraD (2021), decentralized event-triggered FL (2022)	Trigger on weight or gradient change, not on clustering quality; no hard budget guarantee
Kafka for ML	Kafka-ML (FGCS 2022), FL in Kafka-ML (2024)	Neural-network training; Kafka is a transport, not part of the algorithm's recovery story

Gap: no existing method keeps a continuously updated global clustering from non-IID edge streams while provably respecting per-node communication budgets under changing network conditions, using a send trigger tied to the clustering objective, and validated on a real fault-tolerant messaging system.

4. Problem statement and research questions

Setting. There are m edge nodes. Node i sees a stream from its own changing distribution $P_i(t)$, and different nodes see different distributions (some clusters may exist at only one node). Node i keeps micro-cluster summaries $CF = (n, LS, SS, t)$ with time decay, has a link with a changing price $p_i(t)$ (latency, loss, metered bytes), and a byte budget B_i per time window. The coordinator only holds the summaries each node last sent, and from them builds global cluster centres $C(t)$.

Objective. Choose when each node sends so that the global clustering stays as close as possible to what a central learner with all raw data would get, while every node stays within its budget and raw data never leaves a node.

minimise $(1/T) \sum_t [J(C(t)) - J(C^*(t))]$
subject to $(1/T) \sum_t \text{bytes}_i(t) \leq B_i / W$ for every node i

Research questions

RQ	Question
RQ1 Cost vs accuracy	How many bytes does a budget-aware, quality-linked trigger save compared with periodic, change-based and norm-based triggers, at equal clustering quality?
RQ2 Non-IID	Does heterogeneity-aware aggregation keep clusters that only one node sees? How does quality change with the degree of skew?
RQ3 Drift	How fast does the global model recover after sudden, gradual or local-only drift, under a fixed budget?
RQ4 Network	How robust is the system to latency, loss, bandwidth caps and node crashes, and what do Kafka's retention and replay contribute?
RQ5 Scale	How do throughput, end-to-end latency and consumer lag grow with the number of nodes and the stream rate on laptop hardware?

5. Objectives

#	Objective
O1	Design a staleness score, computed locally from micro-cluster summaries, that measures how much the server's copy of a node hurts the global clustering cost.
O2	Design a send rule that meets a byte budget per node, adapts to link conditions, and comes with a proof of budget satisfaction and a bound on the server's cost error.
O3	Design aggregation that down-weights stale data and protects minority and single-node clusters.
O4	Build an open-source, Kafka-native, reproducible testbed that runs on a laptop, with network-fault injection.
O5	Evaluate against at least 6 baselines, on at least 4 datasets and at least 3 non-IID schemes, with at least 5 random seeds and significance tests.
O6	Produce a submission-ready manuscript and a reproducibility package.

6. Methodology

6.1 System architecture

Edge nodes cluster their own streams and publish small deltas to Kafka only when worthwhile. The coordinator consumes them, builds the global model and publishes it back on a compacted topic, which every edge node reads to decide when its next update is worth sending.

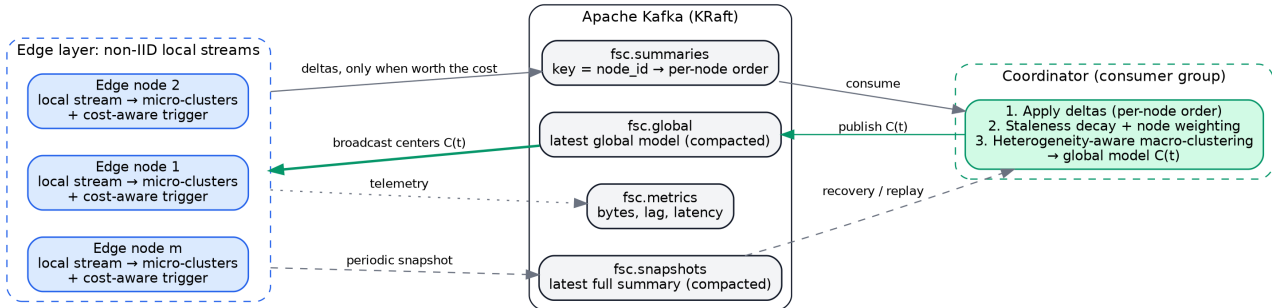


Figure 1. System architecture: edge layer, Kafka topics and coordinator

Kafka topic	Role	Kafka feature used
fsc.summaries	Delta updates from each node	Key = node_id gives per-node ordering; idempotent producer prevents duplicates
fsc.snapshots	Latest full summary per node	Log compaction keeps the latest per key indefinitely, so a crashed coordinator can rebuild its state
fsc.global	Global cluster centres, broadcast to edges	Log compaction keeps only the latest model
fsc.metrics	Bytes, lag, latency telemetry	Used for the evaluation
fsc.raw	Raw points for the centralised baseline only	Upper bound on quality and bytes

6.2 Edge node: when to send (the cost-aware filter)

For each micro-cluster, the k-means cost of assigning it to a centre c has a closed form, $SS - 2 \cdot LS \cdot c + n \cdot \|c\|^2$, so a node can compute exactly how much its unsent changes would move the global clustering cost, without sending any data.

$\Delta = \sum_j |\text{cost drift in global cluster } j| + \rho \cdot (\text{mass shifted between clusters}) + \kappa \cdot (\text{mass far from every global centre})$

send if $\Delta \geq \lambda \cdot p \cdot b$ (b = delta size in bytes, p = link price)

per window: $\lambda \leftarrow \max(\lambda_{\min}, \lambda + \eta \cdot (\text{bytes sent} - B) / B)$

The last term in Δ is a novelty signal: a new cluster seen only at this node is sent quickly, which is the key non-IID case. The multiplier λ rises when a node overspends and falls when it underspends, so the node tracks its budget automatically. A token bucket enforces the budget exactly during bursts.

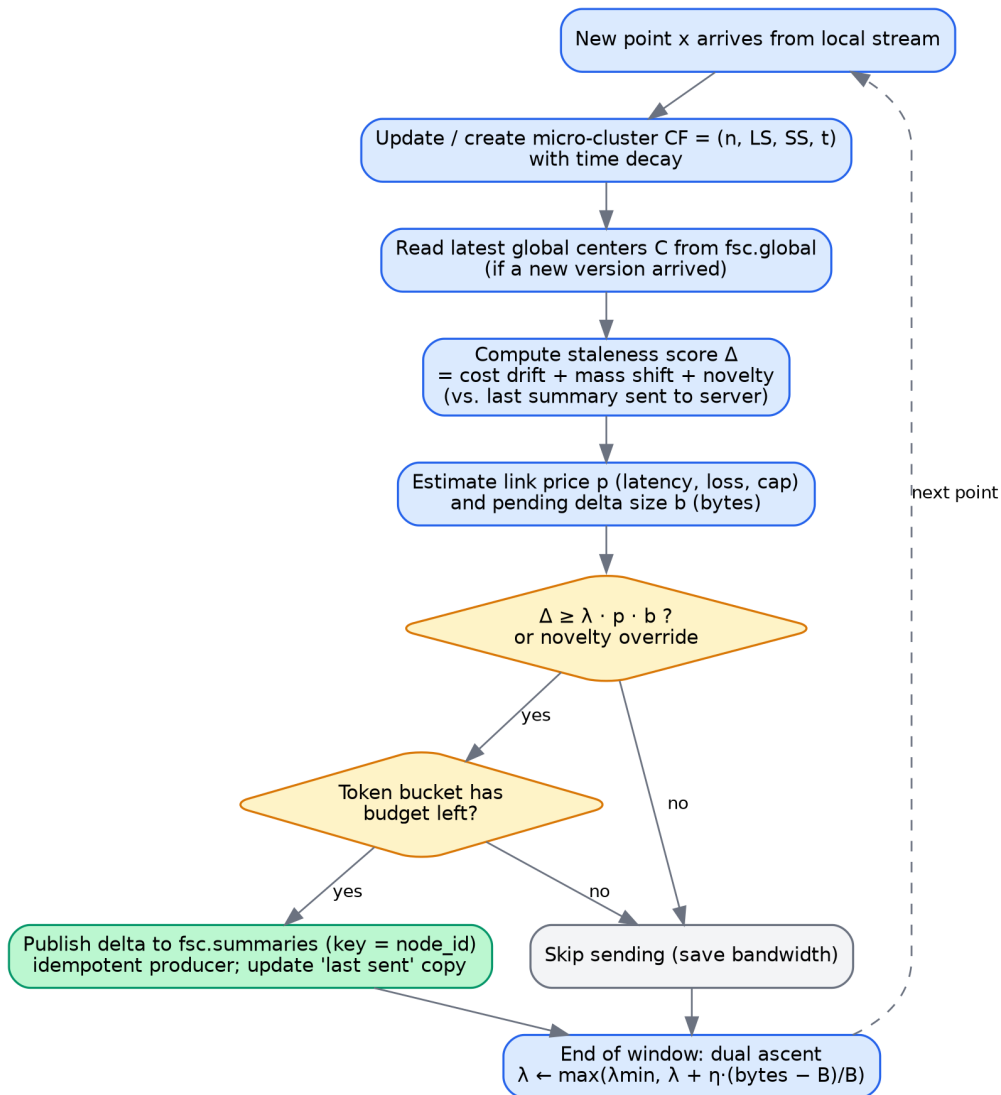


Figure 2. Edge node decision flowchart

6.3 Coordinator: aggregation and recovery

- **Staleness weighting:** every stored summary is decayed to the current time, so nodes that stopped sending fade out instead of dominating.
- **Node balancing:** weight $w = n^\beta$ so one fast node cannot drown out slow ones.
- **Macro-clustering:** weighted k-means++ and Lloyd over micro-clusters, seeded from per-node centres (following k-FED); or weighted DBSCAN when the number of clusters is unknown.
- **Exclusive-cluster protection:** a dense group supported by a single node and far from every centre stays its own cluster.
- **Recovery:** after a crash the coordinator reads the compacted snapshots, then replays deltas from its committed offsets. No summary is lost.

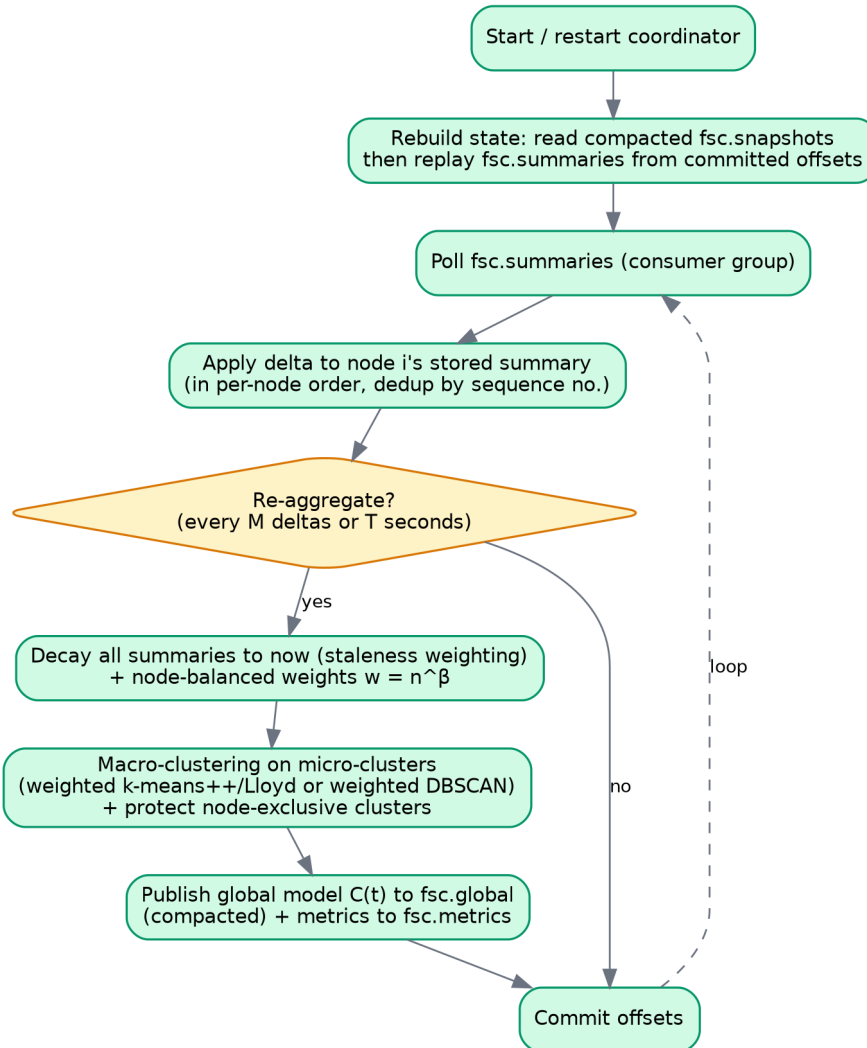


Figure 3. Coordinator flowchart

6.4 Theory (to be proven in week 3)

- **Proposition 1 (staleness bound):** if each node keeps $\Delta_i \leq \tau_i$, the server's estimate of the global cost is off by at most $\sum_i \tau_i$.
- **Proposition 2 (budget):** each node's long-run average bytes stay within B_i/W plus a term that shrinks as $1/(\eta T)$.
- Together: budget $\rightarrow \lambda \rightarrow$ threshold $\rightarrow$ bounded error, an explicit cost-accuracy trade-off.

7. Experimental plan

Datasets (labels used only for evaluation)

Dataset	Why
Synthetic Gaussian / RBF streams with drift	Ground truth; full control of drift and skew
KDD Cup 99 / NSL-KDD	Classic stream-clustering benchmark (known to be dated, so not used alone)
CIC-IDS2017 or UNSW-NB15	Modern network traffic; natural edge-network story
Forest Covertype	Standard stream benchmark with 7 classes
Intel Lab sensors or Electricity (ELEC2)	Real IoT drift with natural per-sensor splits

Non-IID splits

- Label skew with Dirichlet(α), $\alpha \in \{0.1, 0.5, 1, 100\}$
- Cluster-exclusive nodes: each node sees $k' \in \{1, 2, \sqrt{k}, k\}$ clusters
- Quantity skew (power-law rates), drift at only some nodes, and natural per-sensor splits

Baselines

- Centralised-Raw (all points sent; best quality, most bytes)
- Naive-Federated (full summary on every update); Periodic-T (every T seconds)
- Change-Threshold (Tran 2013 / DGClust style); Norm-Event-Triggered (EventGraD style)
- k-FED re-run periodically; plus ablations removing each part of FedCAST

Metrics

Category	Metrics
Quality	ARI, NMI, purity, SSQ ratio to centralised, CMM, recall of minority and single-node clusters
Cost	Bytes on the wire, messages sent, bytes as a share of Centralised-Raw
Timeliness	End-to-end latency, Kafka consumer lag, drift-recovery time
System	Throughput per node, coordinator CPU and memory, crash-recovery time
Statistics	5–10 seeds, 95% confidence intervals, Friedman + Nemenyi, Wilcoxon

8. Expected outcomes

- **Contributions:** a new problem formulation, the FedCAST algorithm, two theoretical results, an open-source Kafka-native system, and an extensive evaluation.
- **H1:** at least 50% fewer bytes than Periodic and Change-Threshold at the same ARI (within 2%).
- **H2:** better recall of single-node clusters than naive aggregation for $\alpha \leq 0.5$.
- **H3:** faster recovery from drift than Periodic at the same budget.
- **H4:** no lost summaries and bounded recovery time after node or coordinator crashes.
- These are hypotheses to test. They will be reported honestly whether or not they hold.

9. Target venues

All journals below accept submissions year-round; review usually takes 2–6 months. Check current quartiles on SJR or JCR before submitting.

Publisher	Venue	Fit
IEEE	IEEE Internet of Things Journal	Edge, IoT and federated systems. Strong fit.
IEEE	IEEE Trans. on Knowledge and Data Engineering	Stream mining with theory; highest bar
IEEE	IEEE Trans. on Parallel and Distributed Systems	If the systems part dominates
Elsevier	Future Generation Computer Systems	Kafka-ML was published here. Very good fit.
Elsevier	Information Sciences; Knowledge-Based Systems	Algorithm focus
Springer	Data Mining and Knowledge Discovery; Machine Learning	Stream-clustering community
Springer	Journal of Big Data; Cluster Computing	Faster review; fallback
ACM	ACM Trans. on Knowledge Discovery from Data	Stream mining
ACM	ACM Trans. on Internet of Things	Edge systems
Conferences	ACM DEBS, IEEE BigData, ECML-PKDD, PAKDD, CIKM, ICDM, ICDE	DEBS is the most natural home for Kafka work; check current deadlines

Recommendation: first submission to Elsevier FGCS or the IEEE Internet of Things Journal; keep IEEE TKDE for a stronger version if the theory is clean.

10. Roadmap and timeline

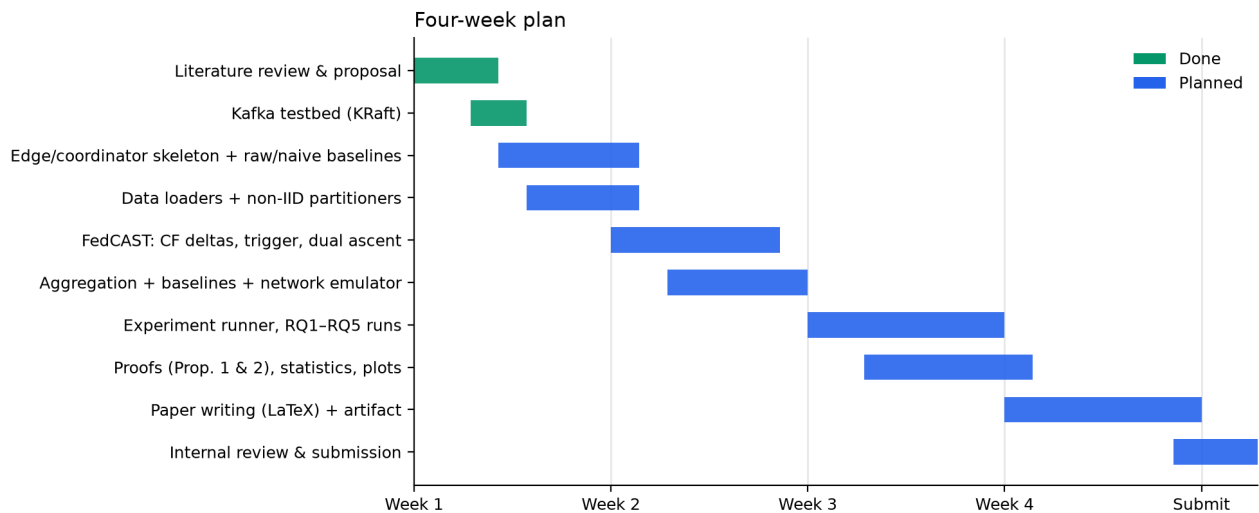


Figure 4. Four-week plan (green = done, blue = planned)

Week	Deliverables
1	Research and proposal ✓; Kafka testbed ✓; edge and coordinator skeleton; raw and naive baselines; data loaders and non-IID splitters
2	FedCAST core: summary deltas, staleness score, budget controller, aggregation; other baselines; in-app network emulator; unit tests
3	Experiment runner; RQ1-RQ5 runs; crash tests; plots; statistics; proofs
4	Paper in the venue's LaTeX template; reproducibility package; review; submission

Decision gates: end of week 1, your team approves the method; end of week 2, a pilot on synthetic data must beat Periodic and Change-Threshold, otherwise we redesign the trigger before scaling up; end of week 3, freeze results and pick the venue.

11. What your team needs to do next

#	Task	Who
1	Read docs/00_research_proposal.md §3-5 and approve or change the direction	Team
2	Check every reference against the original paper (some came from search snippets)	Team
3	Check journal quartiles on SJR / JCR	Team
4	Download datasets that need registration (CIC-IDS2017 / UNSW-NB15) if needed	Team
5	Clear the topic and target venue with your professor	Team
6	Say "go" so Claude starts week-1 coding: edge node, coordinator, baselines, data loaders	Team → Claude
7	Run long experiments on your laptops in week 3 (one command per experiment)	Team

How to run the Kafka testbed now (under 1 GB of RAM)

```
# macOS or Windows (inside WSL2): no Docker, needs Java 17+
scripts/kafka_native.sh start
# Linux: Docker is also fine
docker compose -f docker/docker-compose.yml up -d
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Details, measurements and a Windows WSL2 memory cap: docs/02_low_memory_setup.md

12. Key references

- 1 Aggarwal et al. A framework for clustering evolving data streams (CluStream). VLDB 2003.
- 2 Cao et al. Density-based clustering over an evolving data stream with noise (DenStream). SDM 2006.
- 3 Hahsler & Bolaños. Clustering data streams based on shared density between micro-clusters. IEEE TKDE 2016.
- 4 Cormode, Muthukrishnan & Zhuang. Conquering the divide: continuous clustering of distributed data streams. ICDE 2007.
- 5 Gama, Rodrigues & Lopes. Clustering distributed sensor data streams using local processing and reduced communication. Intelligent Data Analysis 2011.
- 6 Tran. Communication-efficient exact clustering of distributed streaming data. ICCSA 2013 (arXiv:1209.4257).
- 7 Balcan, Ehrlich & Liang. Distributed k-means and k-median clustering on general topologies. NeurIPS 2013.
- 8 Dennis, Li & Smith. Heterogeneity for the win: one-shot federated clustering. ICML 2021.
- 9 Stallmann & Wilbik. Towards federated clustering: a federated fuzzy c-means algorithm. arXiv:2201.07316.
- 10 Zhang et al. Asynchronous federated clustering with unknown number of clusters. AAAI 2025.
- 11 Angelova et al. FedCluLearn: federated continual learning using stream micro-cluster indexing. ECML-PKDD 2025.
- 12 Silva, Vinagre & Gama. Federated anomaly detection over distributed data streams. arXiv:2205.07829.
- 13 Li et al. Density-aware and cluster-based federated anomaly detection on data streams. WSDM 2025.
- 14 EventGraD: event-triggered communication in parallel machine learning. arXiv:2103.07454.
- 15 Decentralized event-triggered federated learning with heterogeneous communication thresholds. arXiv:2204.03726.
- 16 Martín et al. Kafka-ML: connecting the data stream with ML/AI frameworks. FGCS 2022.
- 17 Towards flexible data stream collaboration: federated learning in Kafka-ML. Internet of Things, 2024.
- 18 Epasto et al. Differentially private clustering in data streams. arXiv:2307.07449.
- 19 Neely. Stochastic Network Optimization with Application to Communication and Queueing Systems. 2010.
- 20 Montiel et al. River: machine learning for streaming data in Python. JMLR 2021.